

From Requirements to Release: My Journey Building My First MCP Server

About the Project

- **Project:** My First MCP – To-Do List
 - **Technology:** TypeScript, Model Context Protocol, Zod
 - **Application:** Task Management Dashboard
 - **Deployment:** Render
 - **Repository:** GitHub
-

Introduction

When I first started the MCP training, I was introduced to a technology that was completely new to me. At first, I was mainly trying to understand what MCP is, how it works, and how it can connect AI models with different tools.

For the project, I decided to build a task management MCP server. The idea was simple at the beginning: create tools that allow an AI to work with tasks. But as I continued with the training, the project became much bigger than the first version.

I went through the whole process step by step, starting with the requirements, then building the tools, adding validation and security, testing everything, working on the web dashboard, documenting the project, and finally preparing it for release.

This is the story of how the project developed and what I learned from building it.

Starting With the Requirements

The first thing I had to do was understand exactly what was required.

I started by breaking the project into smaller parts instead of trying to build everything at once. The first tools I worked on were `add_task`, `list_tasks`, and `complete_task`.

At this point, I was still getting used to MCP and how its tools work. I had to think about what information each tool should receive, what it should return, and how an AI client would use it.

This helped me understand that the project was not just about writing a few functions. Each tool needed to have a clear purpose and predictable behavior.

Building the MCP Tools

After understanding the basic requirements, I started building the MCP server using TypeScript.

I first implemented the main task operations. `add_task` was used to create a new task, `list_tasks` was used to view tasks, and `complete_task` was used to mark a task as completed.

I also worked on the input schemas using Zod. This was important because I wanted the server to check the input before processing it and reject invalid data instead of allowing unexpected values into the application.

Later, I expanded the project by adding `update_task` and `delete_task`. With these additions, the project could handle the main task operations: adding, listing, updating, completing, and deleting tasks.

While working on these tools, I started to understand MCP better because I could actually see how an AI client could interact with functionality that I had built myself.

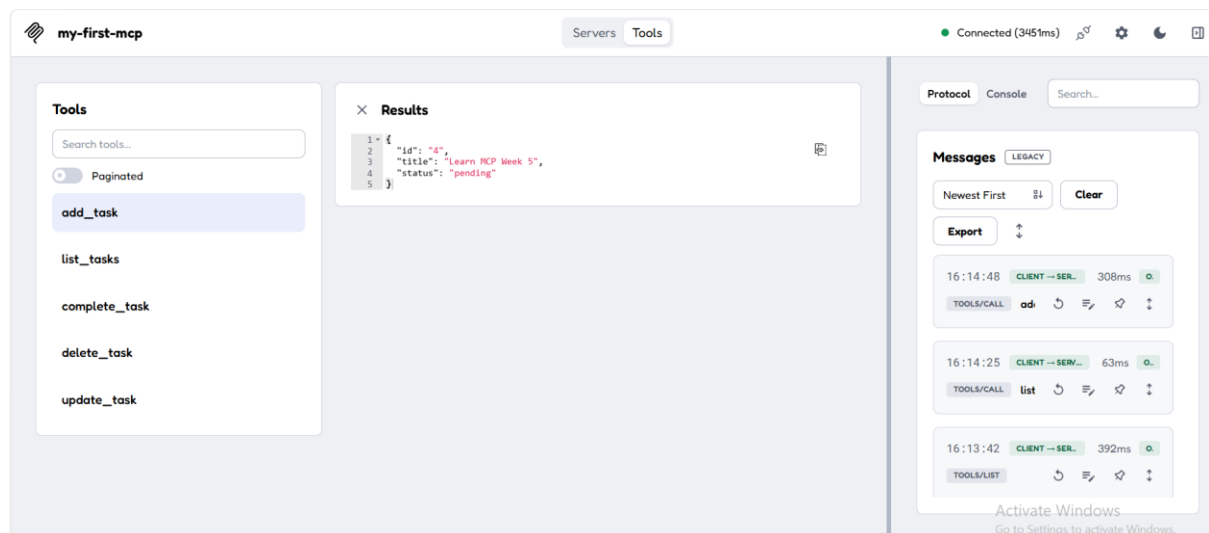


Figure 1: MCP Inspector showing the available task management tools and a successful `add_task` execution

Working With the Data

The next step was making the task data persistent.

I used a JSON file to store the tasks so that the information would not disappear after every operation. The different tools could then work with the same data.

This also introduced some new problems that I had to think about. Since the application was reading and writing a file, I needed to make sure that the correct file was being accessed and that the tools could not access files outside the intended location.

This was one of the points where I started thinking more about what could happen if the application received unexpected input.

Security and Validation

After getting the main functionality to work, I started focusing more on security.

I improved the input validation using Zod and worked on protecting the application from different possible problems. One of the important issues I looked at was path traversal because the project works with a local data file.

I created a threat model to identify the main risks in the project and then worked on possible mitigations.

I also worked on network-related protections, including an HTTP allowlist and timeout handling.

This part of the training changed how I looked at development. Before this project, I would usually focus first on whether a feature works. During this project, I started asking another question as well: what could go wrong if someone gives the application unexpected input?

Testing and Debugging

After implementing the features, I started testing the project.

I created a test plan with different scenarios to check both successful operations and invalid inputs. I used the MCP Inspector to test the tools and make sure they were behaving as expected.

Some tests passed immediately, while others needed more investigation. For example, I had a validation case related to the task status that could not be submitted through the Inspector interface in the way I originally expected.

I also had an issue when connecting the MCP server with Claude Desktop. The server was looking for the task data file in the wrong working directory. After checking the error and tracing where the application was looking for the file, I fixed the configuration.

These problems were actually useful learning experiences. Instead of just seeing a failed test as a problem, I learned to use it to understand more about how the application was working.

Building the Task Dashboard

I also wanted the project to have a proper interface instead of only interacting with it through MCP tools.

I worked on a Task Management Dashboard where the tasks could be viewed and managed through a web interface.

The dashboard shows information such as the total number of tasks, pending tasks, completed tasks, and the completion rate. It also includes the task list and the main actions for managing tasks.

The interface made the project feel more complete because I could see the task system working as an actual application rather than only as a server running in the background.

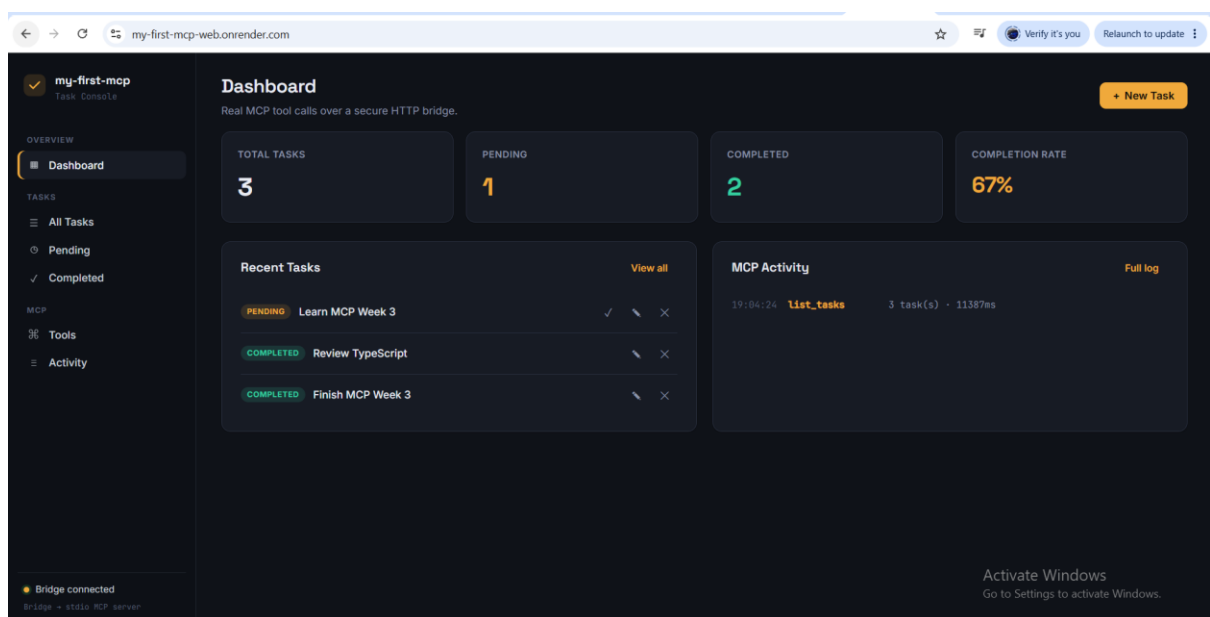


Figure 2: The final Task Management Dashboard showing the task overview and management interface.

Testing the CRUD Operations

As the project became more complete, I also tested the main CRUD operations together.

I tested adding, updating, deleting, and listing tasks and then verified the results. The final CRUD verification completed successfully without errors across the tested operations.

One thing I paid attention to during the testing was making sure that the original tasks were not accidentally changed by the new operations.

The final test gave me more confidence that the tools were working correctly together and not only working individually.

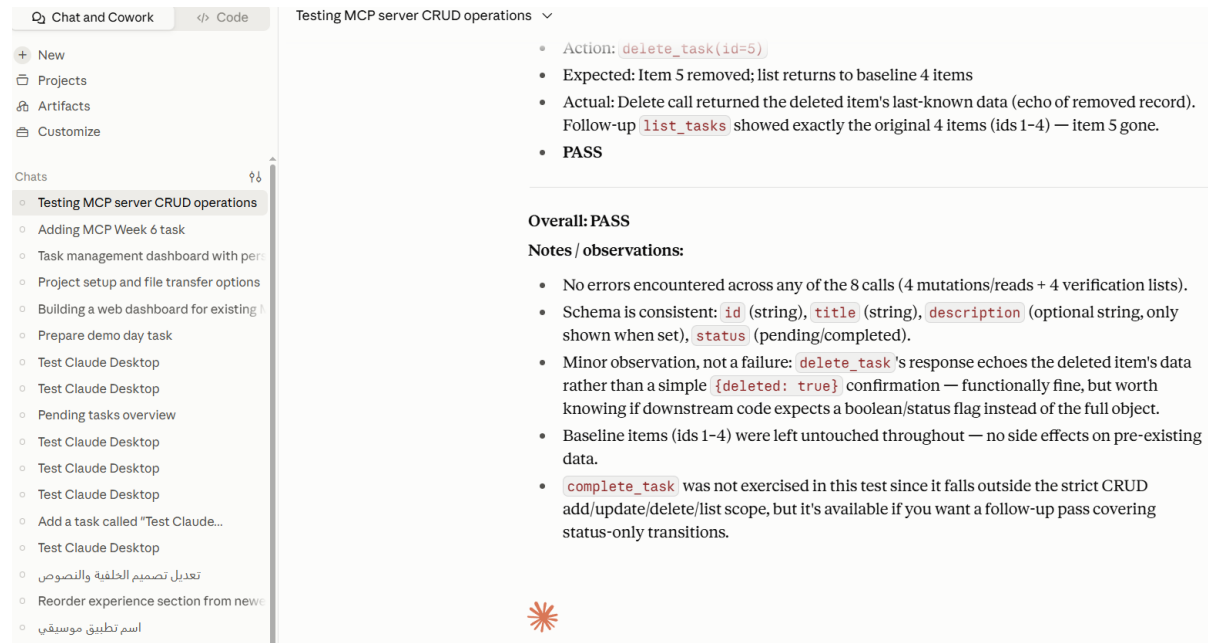


Figure 3: CRUD verification through the AI client showing an overall PASS result

Documentation and GitHub

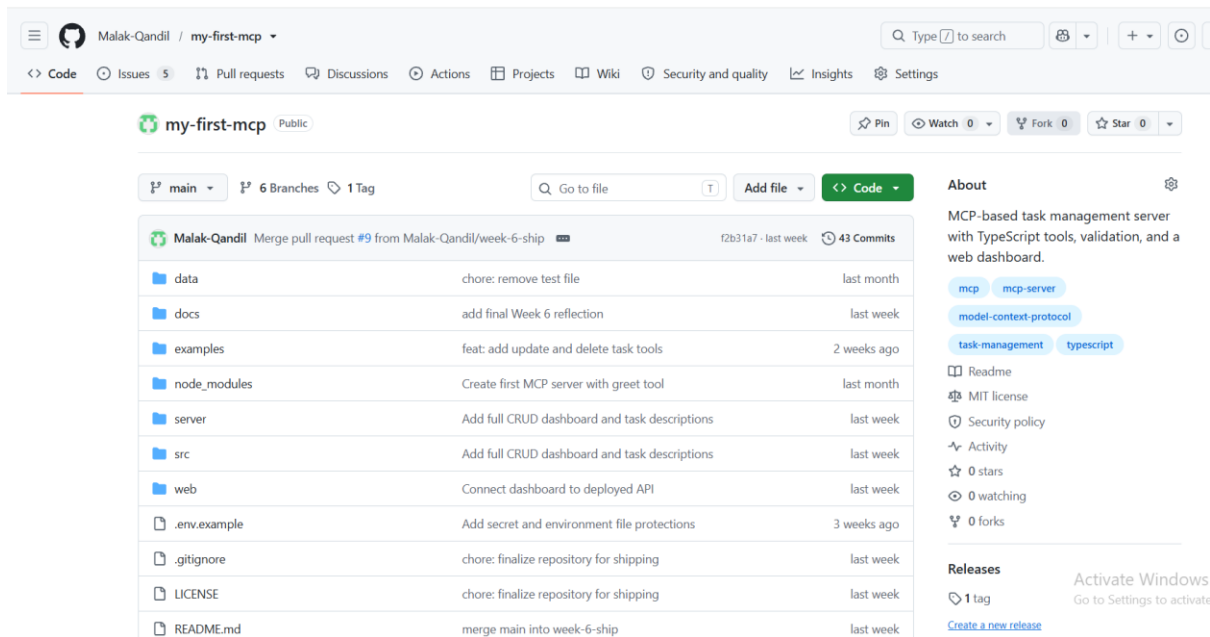
Another important part of the project was documentation.

I worked on the README so that someone who has never seen the project before could understand what it does, what is needed to run it, how to start the server, how to use the MCP Inspector, and what tools are available.

I also created documentation for the threat model, testing, and project review.

Git and GitHub were part of the development process from the beginning. I used branches to organize the work and eventually prepared the repository for public release.

This taught me that completing a project does not mean only finishing the code. If other people are going to use or review the project, they also need clear documentation and an organized repository.



The Challenges I Faced

The biggest challenge for me was working with something that was new to me.

At the beginning, I had to spend time understanding MCP itself before I could properly understand how to build the project. I also faced different issues while testing, debugging, integrating the server, and working with the web interface.

Sometimes I knew what I wanted the project to do, but I did not immediately know how to implement it. I had to search, test different solutions, understand the errors, and try again.

Another challenge was keeping track of all the different parts of the project. It was not only the MCP server. There was also validation, security, testing, documentation, GitHub, and the dashboard.

The biggest thing I learned from this is that a project usually does not become good in one attempt. You build something, find a problem, fix it, test it again, and keep improving it.

What I Learned

This project taught me much more than just how to build an MCP server.

I learned how to work with a new technology from the beginning and how to break a large requirement into smaller tasks.

I also became more comfortable with debugging. When something did not work, I learned not to immediately assume that the whole project was wrong. Instead, I started checking the error, understanding what caused it, and fixing one problem at a time.

Security was another important lesson for me. I learned that validation and security should be considered while building a project, not added only at the very end.

I also learned the importance of testing and documentation. A project can work on your computer, but that does not automatically mean it is ready for other people to use.

Reaching the Final Version

By the end of the training, the project had developed significantly from its first version.

It started with a few basic MCP tools and gradually became a complete task management project with multiple tools, validation, security improvements, testing, documentation, a web dashboard, and AI client integration.

Looking back, I can see that every stage added something important to the final project. Even the problems and failed tests helped me understand the project better.

I am happy with how the project developed and with what I was able to learn throughout the process. Most importantly, this experience gave me more confidence to work with technologies that I have not used before.

Final Thoughts

Building my first MCP server was challenging, but it was also one of the most useful hands-on experiences I had during the training.

I started the project mainly trying to understand MCP, and I finished it with a much better understanding of how AI can interact with tools and applications.

The project also improved my skills in development, debugging, testing, security, documentation, and working with GitHub.

For me, the biggest lesson was that learning a new technology does not happen by understanding everything before starting. Sometimes the best way to learn is to start building, face problems, solve them, and improve the project along the way.

This project was a big step in my learning journey, and I am proud of the progress I made from the first requirements to the final version.

Project Links

GitHub Repository:

<https://github.com/Malak-Qandil/my-first-mcp>

Live Website:

<https://my-first-mcp-web.onrender.com/>

Training Program:

<https://nextflows.ai/academy>